Master PRD — Claude Skill System for a Done-for-You Local Business Website Builder

Copy-paste ready project context document

1. Project Overview

This project is a modular Claude skill system designed to generate high-quality, near-fully customized websites for local service businesses through a done-for-you agency workflow.

The goal is not to build a generic DIY website builder for end customers. The goal is to build an internal AI-assisted production system that lets the agency generate professional client websites without having to manually rethink structure, layout, architecture, feature selection, and frontend decisions each time.

This system should act like a disciplined website production framework. It should ask the right questions, create the right planning outputs, route work to the correct skill folders, and only begin coding once the site is clearly defined.

The final websites should look custom, polished, trustworthy, and premium, while avoiding the obvious “AI-generated” or mass-template feel that makes many generated websites feel fake or low-trust.

2. Core Purpose

The system exists to help produce websites for local service businesses faster and more consistently without sacrificing design quality.

It should:

- generate strong frontend-first websites
- reduce repeated manual planning work
- enforce a consistent production workflow
- create clearer, more professional design outcomes
- support multiple feature types without forcing all features into every build
- keep backend complexity minimal unless truly needed

This is primarily a frontend-oriented system. Visual experience, clarity, trust, and conversion quality matter more than raw development speed.

3. Product Type

This is a done-for-you agency system.

The agency, not the client, is the main operator. The agency provides business information, desired features, and some design direction. The AI system then handles the rest of the website-generation workflow in a structured way.

This is not intended to be:

- a customer-facing no-code builder
- a drag-and-drop SaaS editor
- a client self-serve website platform

The agency will edit and manage the websites. Clients are not the primary editors.

4. Target Users

Primary User

The primary user is the agency operator using Claude to generate websites and related planning outputs.

End Customer Type

The end customers are local service businesses, including but not limited to:

- home services
- clinics
- salons
- consultants
- agencies
- fitness businesses
- other small local service providers

The system should be flexible enough to support a wide range of local-service business types.

5. Business Model Context

The system is being built to support one-time website builds, with recurring revenue potentially coming from things like API management or related ongoing service layers.

This means the generated websites must be:

- high-quality enough to justify a premium done-for-you service
- structured enough to be maintained efficiently
- clear enough to revise later without redoing everything

6. Main Product Goals

The system should optimize for the following priorities, in this exact order:

1. Design quality
2. Ease of editing
3. Code quality
4. Low cost
5. Conversion
6. Scalability
7. SEO
8. Speed

Speed of delivery is the least important priority. The system should not rush into code. It should prioritize better decision-making and better outcomes.

7. Design Philosophy

The visual direction should feel:

- local
- minimal
- premium
- trustworthy
- professional
- polished

The closest reference feeling is Stripe, specifically in terms of:

- clean structure
- restrained sophistication
- strong layout hierarchy
- clear sectioning
- premium but controlled design
- professional visual rhythm

The system may borrow principles and useful structural ideas from strong reference sites, but it should not directly clone them.

8. Anti-AI Design Requirement

A major goal of this project is to prevent websites from looking obviously AI-generated.

The system must avoid common generated-site patterns such as:

- over-glossy gradients everywhere
- neon purple/blue “tech” color palettes when they do not fit the brand
- generic SaaS blobs and abstract floating background shapes
- excessive glassmorphism
- too many animations stacked together
- overly symmetrical template layouts that feel mass-produced
- fake-looking stock photography
- generic vector illustration styles
- too many cards in every section
- giant border radius on everything
- weak typography hierarchy
- too many mixed font styles
- vague buzzword-heavy layouts and copy
- fake dashboards or fake product UI
- overcrowded hero sections
- shadows on every element
- meaningless icon rows
- too many accent colors
- identical spacing everywhere with no rhythm
- fake testimonials or generic trust blocks
- trying too hard to look modern instead of trustworthy

The system should create websites that feel intentionally designed, not auto-generated.

9. Product Scope

The system should support the generation of full business websites, not just simple landing pages.

These may include:

- homepage
- services pages
- about page
- contact page

- location/service-area pages
- optional blog/resources
- optional booking/payment flows
- optional quote forms and lead systems
- optional gallery/case study pages
- optional ecommerce or gated areas when relevant

The websites should be near-fully customized rather than rigidly template-based.

10. Key Workflow Requirement

The system must always follow a strict top-down workflow before coding begins.

Required sequence:

1. Questions
2. Plan
3. Sitemap
4. Wireframe Plan
5. Design System
6. Build Plan
7. Code

This sequence should always be enforced.

The system should not jump directly into coding unless the earlier stages are complete and internally consistent.

11. Workflow Philosophy

Questions

The system should ask clarifying questions first, especially when the request is vague.

Plan

The system should define:

- business context
- site goals
- trust goals

- user actions
- core priorities
- design direction
- feature requirements
- constraints

Sitemap

The system should determine page structure and main navigation logic.

Wireframe Plan

The system should determine section ordering, information hierarchy, content flow, and layout logic before visuals are finalized.

Design System

The system should translate visual direction into concrete rules, not vague labels. This includes:

- typography tone
- spacing rhythm
- button style
- CTA treatment
- image treatment
- color logic
- section density
- animation level

Build Plan

The system should define what will be built, what stack assumptions apply, what components and features are needed, and what complexity should be avoided.

Code

Only after the earlier stages are complete should implementation begin.

12. Technical Philosophy

This is a frontend-first system.

The websites should prioritize:

- strong UI/UX

- clarity
- trust
- responsive quality
- modular code
- easy later editing

Backend

Backend should be minimal by default.

Use backend only when needed for things like:

- forms
- booking
- payments
- chat
- data collection
- CMS
- gated access
- dynamic integrations

CMS

CMS should only be included when specifically requested.

By default, assume the agency will edit the website rather than the client.

13. Stack Direction

The system should assume a strong modern default stack suitable for high-quality website generation.

The stack should generally lean toward a modern frontend framework setup with strong UI control and maintainability. Backend decisions should remain lightweight and conditional.

Minimal backend patterns may include options like Supabase or Firebase where useful, but backend should never become the center of the project.

The stack should support:

- premium frontend output
- mobile-first development
- modular components
- scalable structure

- clean maintainable code

14. File and Skill System Philosophy

This project should be organized as a modular Claude skill system with many focused folders rather than one giant prompt.

The system should contain roughly 20 major folders, each handling a specific job or feature area.

Examples include:

- orchestrator
- discovery
- planning
- sitemap
- wireframes
- design system
- brand direction
- copy structure
- frontend architecture
- backend architecture
- hero
- navigation
- forms
- social proof
- services
- gallery
- booking/payments
- local SEO
- QA/review
- revisions
- PRDs

Each major folder should contain multiple markdown files rather than one bloated file.

Each markdown file should stay under roughly 500 lines and remain tightly scoped.

15. Folder Structure Principle

Each folder should represent one clear functional responsibility.

Each file inside should represent one focused instruction layer.

Example logic:

- folder = one job
- file = one specific set of rules, examples, checklists, or standards

This keeps the system readable, maintainable, and easier to improve over time.

16. Orchestrator Role

The `00-orchestrator` folder should act as the master controller for the whole system.

Its role is to:

- enforce workflow order
- ask clarifying questions
- route tasks to the correct skill folders
- activate the right feature folders for each website
- prevent coding too early
- preserve unaffected areas during revisions

It should not directly do design-system work itself. It should delegate that work to the relevant folders.

It is responsible for decision-making and flow control, not detailed execution.

17. Visual Reference Process

Before style direction is finalized, the system should ask for 2–3 visual reference websites whenever possible.

These references should be used to extract principles, not to blindly copy.

Visual inspiration must be translated into concrete design instructions such as:

- typography tone
- layout density
- CTA style
- image treatment
- spacing logic
- animation level

The system should avoid vague labels like “modern” or “premium” without turning them into practical rules.

18. Website Feature Support

The system should be capable of supporting all major local-business website features, but it should not force all of them into every project.

Supported feature types include:

- clear hero section
- sticky navigation and header
- mobile-first design
- contact form
- click-to-call button
- booking and appointment scheduling
- online payments and deposits
- quote and estimate request forms
- lead capture sections or popups
- live chat or chatbot
- testimonials and reviews
- case studies and success stories
- gallery and portfolio
- before-and-after slider
- services and products pages
- pricing and package section
- FAQ section
- map, location, and service-area section
- multi-step forms
- email newsletter signup
- social media integration
- Google Reviews embed
- trust badges, certifications, and partner logos
- CMS when requested
- blog and resources section
- multilingual support
- membership and client portal

- ecommerce, cart, and checkout
- analytics and conversion tracking
- SEO basics

Each of these feature areas can be represented as its own modular skill folder or module set.

19. Revision Philosophy

Revisions are important and should be treated as a first-class part of the system.

When revisions are requested, the system should:

- change only the affected area when possible
- preserve the rest of the architecture
- avoid unnecessary rewrites
- maintain consistency with the rest of the design system
- support future improvement cycles such as refinement, optimization, SEO upgrades, or expansion

20. Copy and Asset Assumptions

The agency will generally provide:

- logos
- images
- business details
- service information
- client-specific content later

The system does not need to invent all final content upfront, but it should still provide:

- content structure
- formatting expectations
- layout placeholders
- image-placement logic
- visual treatment guidance

This ensures the site structure and design system are still solid even before all final content is attached.

21. Mobile-First Requirement

Mobile-first design is non-negotiable.

Every generated website should be designed and built with mobile usability as a core assumption, not as an afterthought.

Navigation, CTA placement, form usability, spacing, and hierarchy should all be optimized for smaller screens first.

22. Quality Standard

The websites produced by this system should feel:

- custom
- intentional
- clean
- premium
- trustworthy
- easy to understand
- easy to navigate
- visibly higher quality than generic website templates

The frontend experience matters most because that is what the client sees first.

23. Non-Goals

This project is not primarily trying to:

- maximize speed above all else
- create a generic SaaS-style builder for end users
- create overly complex backend systems
- create flashy, trend-chasing design
- generate bloated frontend experiences
- rely on visual gimmicks to create perceived quality

24. Success Criteria

The project is successful if it can consistently help produce websites that:

- look professionally designed rather than AI-generated
- follow a reliable top-down workflow
- ask the right questions before building
- produce strong planning outputs before code

- route work to the correct skill areas
- remain modular and maintainable
- stay frontend-first and minimally complex
- support multiple local business types
- preserve trust and clarity across the whole site

25. Deliverables for the Full System

The final project should eventually include:

- a modular folder structure with core workflow folders
- feature-specific skill folders
- `00-orchestrator` and other high-level controller files
- detailed frontend PRD
- medium-depth backend PRD
- design-system rules
- anti-AI design rules
- workflow and routing logic
- revision handling rules
- QA and review checklists
- examples and templates inside focused markdown files

26. Final Summary

This project is a structured Claude-based internal production system for building premium, trustworthy, near-fully customized websites for local service businesses.

It is not trying to be a flashy all-purpose builder. It is trying to be a disciplined website generation framework that produces better outcomes through:

- better questions
- better planning
- stronger visual discipline
- modular workflow
- restrained frontend decisions
- minimal backend complexity
- and a very high standard for avoiding generic AI-looking design